

Multi-Label Learning with Emerging New Labels

Yue Zhu^{ID}, Kai Ming Ting, and Zhi-Hua Zhou^{ID}, *Fellow, IEEE*

Abstract—In a multi-label learning task, an object possesses multiple concepts where each concept is represented by a class label. Previous studies on multi-label learning have focused on a fixed set of class labels, i.e., the class label set of test data is the same as that in the training set. In many applications, however, the environment is dynamic and new concepts may emerge in a data stream. In order to maintain a good predictive performance in this environment, a multi-label learning method must have the ability to detect and classify instances with emerging new labels. To this end, we propose a new approach called Multi-label learning with Emerging New Labels (MuENL). It has three functions: classify instances on currently known labels, detect the emergence of a new label, and construct a new classifier for each new label that works collaboratively with the classifier for known labels. In addition, we show that MuENL can be easily extended to handle sparse high dimensional data streams by simply reducing the original dimensionality, and then applying MuENL on the reduced dimensional space. Our empirical evaluation shows the effectiveness of MuENL on several benchmark datasets and MuENLHD on the sparse high dimensional Weibo dataset.

Index Terms—Multi-label learning, incremental learning, emerging new labels, learnware

1 INTRODUCTION

IN traditional supervised learning, one instance is associated with a single label. Yet, in many applications, one instance may possess multiple labels. For example, a scene image is usually annotated with several tags [1]; a document may hold multiple topics [2]; and a piece of music may belong to different genres [3].

Multi-label learning is the learning paradigm to handle such kind of data, and has attracted much attention in recent years [4]. Previous studies on multi-label learning have focused on a fixed set of class labels. That is, they assume that the test data have the same set of class labels as that of the training data. In many real-world data mining tasks, however, the environment is dynamic. We study a dynamic scenario in which new labels may emerge together with known labels in an observed instance of a data stream.

In a dynamic environment, a learning system must be able to reuse a previously learned model as well as to adapt the model to the changing environment [5]. In the multi-label learning setting, the system must be able to revise a pre-trained model as new instances are observed; and new classifiers are established for all emerging new labels. These demands are non-trivial, and no existing systems in the literature can meet these demands, as far as we know.

Under the dynamic multi-label learning setting, we assume that instances arrive in a data stream, and no ground truths for class labels are available in the data stream at all times, except for the initial training set. This can be regarded

as a special weakly supervised learning [6]. As a result, detecting and modeling new labels are the key challenges. Specifically, the most difficult part is to detect instances with any new label. Since we do not have any prior knowledge of the new label and it almost always co-occurs with some known labels, it is very difficult to separate instances with new labels from those with the known labels only.

Moreover, because the detection is not perfect, the error will accumulate as more and more new labels emerge in a data stream. Thus, the environment demands robust models in order to maintain a high detection and prediction performances continuously in a data stream, which is also a challenging task.

To meet all the above challenges, we propose a novel Multi-label learning with Emerging New Labels (MuENL) approach to address the dynamic multi-label learning problem. MuENL consists of three components: (1) A classifier is built to optimize both the pairwise label ranking loss and the classification loss on the known labels; (2) a specially designed detector based on both the input features and predicted label attributes; and (3) a classifier updating process that incorporates detected new labels to produce a robust classifier which can tolerate detection errors, and remodels the detector for each new label identified.

The central idea of this paper is to regard instances with emerging new labels as outliers to the norm—instances of known labels seen thus far. This admits outlier detection methods to be used in the dynamic multi-label learning problem. We show that the idea works in practice.

In addition to addressing the core challenges in dynamic multi-label learning problem, we also propose an extension to deal with sparse high dimensional data streams. In a social network site such as Weibo, users post short-text messages with diverse topics. In a bag-of-words representation, each dimension represents a word in a dictionary of thousands of words. Each message is thus a sparse representation of a high dimensional space. We show that MuENL can be easily extended to MuENLHD to handle sparse high dimensional data streams.

- Y. Zhu and Z.-H. Zhou are with the National Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China.
E-mail: {zhu, zhouzh}@lamda.nju.edu.cn.
- K.M. Ting is with the School of Engineering and Information Technology, Federation University, Churchill, VIC 3350, Australia.
E-mail: kaiming.ting@federation.edu.au.

Manuscript received 26 June 2017; revised 13 Feb. 2018; accepted 18 Feb. 2018. Date of publication 1 Mar. 2018; date of current version 10 Sept. 2018. (Corresponding author: Yue Zhu.)

Recommended for acceptance by J. M. Phillips.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TKDE.2018.2810872

The contribution of this work is summarized as follows:

- Formalizing the dynamic multi-label learning problem, which is different from previous multi-label learning settings: new labels may emerge with arriving new instances.
- Proposing the MuENL approach to address the dynamic multi-label learning problem. It has an accurate detector for new labels and a robust classifier for both known and new labels. The time complexity of each component is analyzed. We also examine one way to handle multiple new labels which emerge at the same time using a meta label.
- Extending MuENL to MuENLHD to handle sparse high dimensional data streams.
- Extensive empirical studies are conducted to validate the effectiveness of our approach.

The rest of paper is organized as follows: Section 2 describes several related works on multi-label learning, incremental learning and outlier detection. Section 3 introduces the problem formulation and the MuENL approach. Section 4 extends MuENL to MuENLHD to handle sparse high dimensional data streams. Section 5 analyses the time complexity. Section 6 and Section 7 describe the experimental studies on (dense) low/medium dimensional data streams and sparse high dimensional data streams, respectively. The conclusions are given in the last section.

2 RELATED WORK

Multi-label learning can be divided into three main categories based on the order of label correlations [4]. For the first-order strategy, none of the label correlations are considered. BR [1], for example, trains a classifier for each label independently. For the second-order strategy, pairwise label relations are taken into account. In this strategy, CLR [7] transforms the multi-label learning problem into a pairwise label ranking problem. For the high-order strategy, a label is assumed to be influenced by all other labels. CC [8], for instance, transforms the multi-label learning problem into a chain of binary classification, where the ground-truth labels are successively encoded into the feature space. All the above multi-label learning approaches assume that the class label set is fixed and do not admit new labels. As such, they cannot handle the dynamic multi-label learning problem we investigated in this paper.

Incremental learning is critical for the tasks where a frequent data update is involved or when it is desirable not to re-train the model from scratch. According to [9], there are roughly three major incremental learning settings, i.e., example incremental learning (E-IL) [10], where new instances arrive after the learning system has been trained; attribute incremental learning (A-IL) [11], where new features may appear; and class incremental learning (C-IL) [12], [13], [14], where the class label set may be enlarged. Learning with new labels is a kind of C-IL, which has been studied under various names including zero-shot learning [15], [16] and open-set classification [17], [18]. Besides, the basic assumption of those C-IL works is that the class labels on each instance are mutually exclusive, namely, they are in a multi-class learning setting.

The dynamic multi-label learning setting with emerging-new labels is a combination of E-IL and C-IL, where a new instance may be associated with multiple new labels that co-occur with known labels. A straightforward approach to

adapt C-IL under our setting is to transform the multi-label learning into the multi-class learning by converting each possible label combination into a class [19]. Unfortunately, this approach has two severe limitations. First, a new class may not correspond to a new label, but an unseen combination of known labels. Second, when the label set is large, the number of possible classes is huge. This leads to a difficult training problem, i.e., having an extremely small number of positive instances for most classes. As a result, it cannot be applied in practice.

When a part of the dynamic multi-label learning problem is converted to be an outlier detection problem, many existing methods can be applied; but not in a straightforward manner. For example, OC-SVM[20] learns a boundary for instances with known labels, and decides instances outside the boundary as outliers; iForest [21] predicts instances located in a sparse region as outliers. However, under the multi-label setting, a new label may co-occur with known labels, which makes it difficult to separate instances with new labels only from instances with known labels only.

Recently, Fu [22] has proposed a transductive multi-label zero-shot learning. However, in a transductive setting, all test instances are assumed to be available during the training and all the new labels are assumed to be known. As a result, it cannot be applied in our setting—new instances successively arrive, and we do not know when one or more new labels may occur; or the total number of new labels may occur in one time period.

Another line of related works is multi-label learning with missing labels. Many approaches aim to recover missing labels by exploiting low rank structure or label correlations [23], [24], [25], [26]. In their settings, the labels, which are missing in some instances, are within the observed class label set. In other words, they assume no previously unseen labels; and the missing labels cannot be recovered if they have never been observed during training. Therefore, these approaches cannot deal with novel labels in our setting.

3 THE MUENL APPROACH

The dynamic multi-label learning problem we studied in this paper faces the following challenges: (A) detecting instances with emerging new labels which are also associated known labels; (B) building a robust classifier for both the new and known labels. In this section, we propose an approach called MuENL to handle the dynamic multi-label learning problem that addresses the two challenges.

3.1 Problem Formulation

In open dynamic multi-label learning, we have an initial labeled training set, then unlabeled data successively arrive in a streaming fashion. Let $\mathcal{X}$ denote the feature space and define $X_0 = [x_{-n+1}, \dots, x_{-1}, x_0]^T \subseteq \mathcal{X}$ as the observed n labeled instances in the initial labeled training set. The arriving data stream has an unlabeled instance x_t observed at time t . Let $X_t, t \in \{1, 2, \dots, T\}$ be the accessible data trunk at time t .

We denote by $v_0 = \{1, 2, \dots, \ell\}$ the known class label set¹ at $t=0$, which is the union of all distinct labels appearing in the initial training set. Let v_t be the class label set at time t , and its maximum indexed label is ℓ , which equals to the length of v_t . When a new label is to

1. The number in v represents the index of the label names.

be converted to a known label at time t , the class label set will be enlarged with a new label $\ell' = \ell + 1$: $v_t = v_{t-1} \cup \{\ell'\}^2$; otherwise, it will not change: $v_t = v_{t-1}$. Let $Y_0 = [y_{-n+1}, \dots, y_{-1}, y_0] \in \{-1, 1\}^{\ell \times n}$ be the known label matrix of X_0 in the initial training set, and $y_t = [y_{t,1}, \dots, y_{t,\ell}] \in \{-1, 1\}^\ell$ be the label vector of the newly arrived instance x_t , where $y_{t,j} = 1$ suggests that x_t holds the j -th label; $y_{t,j} = -1$ otherwise. Note that none of the elements in y_t are observed, since x_t is unlabeled for $t > 0$. The dynamic multi-label learning problem is defined as:

Definition 1 (Dynamic Multi-Label Learning). Given X_t and Y_0 , the task is to learn a function set $\mathcal{H}_t = [h_{t,1}, h_{t,2}, \dots, h_{t,\ell}]$, where $h_{t,j} : \mathcal{X} \rightarrow \{-1, 1\}^\ell$ represents the classifier for j th class label, $j \in \{1, 2, \dots, \ell\}$, at time $t \in \{1, 2, \dots, T\}$. For each x_t , $\hat{y}_t = \mathcal{H}_t(x_t)$ is the predicted label vector, where x_t may be associated with both new labels and known labels.

3.2 The Approach

Directly estimating $\mathcal{H}_t$ in the dynamic learning environment defined above is extremely hard, since v_t has an unknown variable, i.e., it is not known whether an observed instance x_t has a new label. In addition, we assume that the ground truth is not available throughout the entire data stream. Thus, the ability to accurately identify the new label when it emerges in x_t is critical in not only detecting its emergence, but also in maintaining a highly accurate classifier for the expanded set of known labels throughout the data stream.

We approach the problem by creating a detection function for any previously unseen labels, i.e., $\mathcal{D}_t(x_t)$, which outputs 1 if x_t holds a new label; or -1 otherwise. For each instance x_t in the data stream, the current classifier is used to do the prediction, yielding $\hat{y}_t = \mathcal{H}_t(x_t)$, where $\mathcal{H}_t = [h_{t,1}, h_{t,2}, \dots, h_{t,\ell}, \mathcal{D}_t]$.

Algorithm 1 summarizes the MuENL approach. It has three key components: (i) the multi-label classifier ($\mathcal{H}_t = [h_{t,1}, \dots, h_{t,\ell}]$) for the known labels; (ii) the detector ($\mathcal{D}_t$) for the new label; (iii) Updating $\mathcal{H}_t$ and $\mathcal{D}_t$ to become $\mathcal{H}_{t+1}$ and $\mathcal{D}_{t+1}$.

After making prediction $\hat{y}_t$ for x_t , the classifier and detector update process begins when both of the following conditions are satisfied:

- 1) When $\mathcal{D}_t(x_t) = 1$, the newly arrived instance associated with the new label is added to buffer B .
- 2) $|B|$ reaches the preset maximum buffer size.

The classifier and the detector are updated using $\mathcal{T}_t = (X_t, \mathcal{H}_t(X_t))$ as shown in line 11 of Algorithm 1. This is the time when the known label set is expanded by including the new label $\ell' = \ell + 1$: $v_t = v_{t-1} \cup \{\ell'\}$. Note that, when $0 < |B| < \text{MAX_BUFFER_SIZE}$, even though the new label has been detected, the data is not sufficient to train/update a good performing classifier. In this situation, the output of the detector is applied as the prediction for the new label.

In order to reduce the storage and computational complexity, not all the historical data are stored in X_t , i.e., after the classifier and the detector have been updated for the new class, only a subset of X_t is stored for future processing. Specifically, in order to give preference to recent instances, we maintain a sampling weight for each instance in the trunk, i.e., s_t , which is initialized as 1.0 when x_t is first observed (line 6 in Algorithm 1). Then, s_t is multiplied by a decay factor 0.8 after the classifier and the detector has been

updated for the new class (line 14 in Algorithm 1).³ Finally, a subset of X_t is selected as follows: an instance in X_t is selected only if a randomly generated number is smaller than the instance's weight in s_t (line 15 in Algorithm 1).

Algorithm 1. MuENL

Input: $X_0, Y_0, \{x_t, t \in \{1, 2, \dots, T\}\}$

Output: $\hat{y}_t$ for each x_t

- 1: Train $\mathcal{H}_0$ with X_0 and Y_0 ; build detector $\mathcal{D}_0$ on X_0 ; set $t = 1$;
 - 2: Initialize sampling weight vector $s_0 = \mathbf{1}_{|X_0|}$;
 - 3: $\mathcal{H}_1 = [\mathcal{H}_0, \mathcal{D}_0]$; $\mathcal{D}_1 = \mathcal{D}_0$;
 - 4: **repeat**
 - 5: Receive a new instance x_t , $X_t = [X_{t-1}; x_t^\top]$;
 - 6: Update sampling weight vector $s_t = [s_{t-1}; 1]$;
 - 7: $\hat{y}_t = \mathcal{H}_t(x_t)$, where $\mathcal{H}_t = [h_{t,1}, h_{t,2}, \dots, h_{t,\ell}, \mathcal{D}_t]$;
 - 8: **if** $\mathcal{D}_t(x_t) = 1$
 - 9: Add x_t to B ;
 - 10: **if** $|B| = \text{MAX_BUFFER_SIZE}$
 - 11: Create $\mathcal{D}_{t+1}$ and $\mathcal{H}_{t+1}$ by using $\mathcal{T}_t = (X_t, \mathcal{H}_t(X_t))$;
 - 12: Empty B ;
 - 13: The new label is converted to the known label:
 $\ell \leftarrow \ell + 1$; $v_t = v_{t-1} \cup \{\ell\}$;
 - 14: Update sampling weight vector $s_t \leftarrow 0.8s_t$;
 - 15: $X_t \leftarrow$ Select a subset of X_t based on s_t ;
 - 16: Update s_t based on the updated X_t ;
 - 17: **end if**
 - 18: **end if**
 - 19: $t \leftarrow t + 1$;
 - 20: $v_t = v_{t-1}$; $\mathcal{D}_t = \mathcal{D}_{t-1}$; $\mathcal{H}_t = \mathcal{H}_{t-1}$;
 - 21: **until** $t = T$.
-

Although we have mentioned only one new label in the definition, the algorithm admits multiple new labels to emerge in the same period. In this scenario, the multiple new labels can be treated as a single meta new label for the purpose of prediction of x_t and model update.

In the following three sections, we detail the design of the multi-label classifier, the detector, and their updates.

3.2.1 The Multi-Label Classifier (PLR)

We use a linear classifier (w_i, b_i) for each label i , i.e., $h_i(x) = \text{sign}(w_i^\top x + b_i)$. In addition to the ordinary misclassification loss minimization, we also minimize the pairwise label ranking loss in order to exploit label correlations to obtain a performance better than using either one of them. The resultant classifier is named Pairwise Label Ranking classifier or PLR for short.

The optimization process is an iterative process, conducted for each w_i for label i while having other w_j fixed ($j \neq i$), until it converges. Specifically, the optimization problem for each w_i is formulated as follows:

$$\begin{aligned}
 & \min_{w_i, b_i, \xi, \zeta} \frac{1}{2} \|w_i\|^2 + C_1 \sum_{k=1}^n \xi_k + C_2 \sum_{j=1}^{\ell} \sum_{k=1}^n \zeta_{j,k}, \\
 & \text{s.t. } y_{i,k} f_{i,k} \geq 1 - \xi_k, \\
 & \quad \Delta_{j,k}(f_{i,k} - f_{j,k}) \geq 1 - \zeta_{j,k}, \\
 & \quad \xi_k \geq 0, \zeta_{j,k} \geq 0, \\
 & \quad j \in \{1, 2, \dots, \ell\}, k \in \{1, 2, \dots, n\},
 \end{aligned} \tag{1}$$

3. This factor is set by experience, it is more robust to different datasets, compared with other settings.

where $\Delta_{j,k} = y_{i,k} - y_{j,k}$, $f_{i,k} = \mathbf{w}_i^\top \mathbf{x}_k + b_i$, and C_1, C_2 are two parameters to trade off.

To simplify the optimization process, $(\mathbf{w}_i, b_i)$ is converted to $\mathbf{w}_i$ by adding an attribute with value 1 at the end of $\mathbf{x}_k$, i.e., $f_{i,k} = \mathbf{w}_i^\top [\mathbf{x}_k; 1]$. Eqn. (1) can be rewritten as

$$\min_{\mathbf{w}_i} \sum_{j=1}^{\ell} \sum_{k=1}^n [1 - (y_{i,k} - y_{j,k})(f_{i,k} - f_{j,k})]_+ + \lambda_1 \sum_{k=1}^n [1 - y_{i,k} f_{i,k}]_+ + \frac{\lambda_2}{2} \|\mathbf{w}_i\|^2, \quad (2)$$

where λ_1 and λ_2 are two trade-off parameters. To solve Eqn. (2), we first calculate the subgradient of the objective function; then apply the gradient descent method.

3.2.2 New Label Detection

The appearance of a new label may be due to a **previously unseen set of feature values** or a **previously unseen pattern of known labels** or **both**. Therefore, we take both feature and label spaces into account: If a new instance has different characteristics from known instances in the feature space, it is likely to hold a new label. Also, in the label space, if a rare co-occurrence of label pattern appears, a new label is likely to occur.

Using this idea, we propose MuENLForest as the detector $\mathcal{D}$. Similar to the earlier work [8], [27], we **encode the label information into the feature space**. Once the data is encoded, a detector for new labels is built based on an **effective anomaly detection technique using random trees called iForest** [21]. The main differences between MuENLForest and iForest are listed as follows:

- iForest considers only the feature space, whereas we encode the label information into the feature space in MuENLForest, considering both the feature difference and label relations. Note that the labels of all instances, arrived after the initial training set, are not available. The predicted value vectors are used for all these instances in training a MuENLForest.
- In each node of a tree in iForest, the test attribute is randomly selected; so is the cut-off value. In contrast, each node of a tree in MuENLForest is based on a fixed number of randomly selected attributes. The split is an outcome of a clustering process based on the selected attributes.
- When evaluating a test instance, iForest employs the average path length, that the test instance traverses over all trees, as the anomaly score. A small value suggests that the test instance is located in a sparse region, which is more likely to be an outlier. This does not work in the multi-label setting since instances with new labels may share the same dense region of instances with some common known labels. MuENLForest captures the characteristics in both the feature space and the label patterns. In addition to building the trees, a ball is constructed in each leaf node based on the training instances which fall into the leaf node. A test instance is predicted to have a new label if it falls outside the ball; otherwise, it has similar data characteristics and label patterns as the training instances used to build MuENLForest.

3.2.3 MuENLForest Construction

Recall that the training set at time $t > 0$ is $\mathcal{T}_t = (X_t, \mathcal{H}_t(X_t))$; and $\mathcal{T}_0 = (X_0, Y_0)$. MuENLForest consists of g MuENLTrees; and each MuENLTree is built using a random subset of $\mathcal{T}_t$ of size ψ using sampling weight s_t . The definition of MuENLTree is given as follows.

Definition 2. *MuENLTree is a binary tree consists of internal nodes and leaf nodes. Let $\mathbf{a} = [\mathbf{x}, \mathcal{H}_t(\mathbf{x})]$ denote a training sample with predictive values. Each internal node has test: $\|\mathbf{a}^q - \mathbf{p}_1\| \leq \|\mathbf{a}^q - \mathbf{p}_2\|$ which splits into two son nodes, where $\mathbf{p}_1$ and $\mathbf{p}_2$ are two cluster centers having q attributes and $\mathbf{a}^q$ is the q projection of $\mathbf{a}$. Each leaf node defines a ball covering S (i.e., the set of all training instances falling into this leaf node) having radius $r = \max_{\mathbf{x} \in S} \|\mathbf{a} - \mathbf{m}\|$, where $\mathbf{m} = \text{mean}(S)$.*

To grow a MuENLTree during the training process, the training set is recursively divided as internal nodes are constructed until any one of the following conditions (C) is satisfied: (a) the tree reaches a height limit e_m ; (b) $|S| = 1$; (c) all instances in S have the same $\mathbf{x}^q$ value. Algorithm 1 summarizes the construction of MuENLTree.

Procedure 1. MuENLTree

Input: Training sample S , current tree height e , maximum tree height e_m , number of randomly selected attributes k

Output: MuENLTree

- 1: **if** any condition in C is satisfied:
- 2: A ball⁴ is built having radius $r = \max_{\mathbf{a} \in S} (\|\mathbf{x} - \mathbf{m}\|)$, where $\mathbf{m} = \text{mean}(S)$;
- 3: **return** $N\{N.S \leftarrow S, N.\mathbf{m} \leftarrow \mathbf{m}, N.r \leftarrow r\}$;
- 4: **else**
- 5: Let Q_1 be the input attribute set in S ;
- 6: Let Q_2 be the predicted attribute set in S ;
- 7: Randomly select k attributes $\mathbf{q}_1 \subset Q_1$;
- 8: Randomly select k attributes $\mathbf{q}_2 \subset Q_2$;
- 9: $\mathbf{q} = [\mathbf{q}_1, \mathbf{q}_2]$;
- 10: Cluster centers: $\{\mathbf{p}_1, \mathbf{p}_2\} \leftarrow \text{Clustering}(\mathbf{q}, S)$;
- 11: $S_l = \{\mathbf{a} \in S \mid \|\mathbf{a}^q - \mathbf{p}_1\| \leq \|\mathbf{a}^q - \mathbf{p}_2\|\}$;
- 12: $S_r = \{\mathbf{a} \in S \mid \|\mathbf{a}^q - \mathbf{p}_1\| > \|\mathbf{a}^q - \mathbf{p}_2\|\}$;
- 13: **return** $N\{N.S \leftarrow S, N.\mathbf{m} \leftarrow \mathbf{m}, N.r \leftarrow r,$
 $N.\mathbf{q} \leftarrow \mathbf{q}, N.\{\mathbf{p}_1, \mathbf{p}_2\} \leftarrow \{\mathbf{p}_1, \mathbf{p}_2\},$
 $N.N_{left} \leftarrow \text{MuENLTree}(S_l, e + 1, e_m, k)$
 $N.N_{right} \leftarrow \text{MuENLTree}(S_r, e + 1, e_m, k)\}$;

14: **end if**

For the construction of MuENLForest, we augment each instance with its predictive values, so that both the feature and label information can be considered. Projected on a set of randomly selected attributes, each internal node is split based on a cluster center on either branch. As a result, instances within the same leaf node must be similar on some attributes of either features or predictive values, or both.

3.2.4 MuENLForest Detection

Once MuENLForest, i.e., $\mathcal{D}_t(\cdot)$, is constructed, it is ready for prediction. In evaluating a test instance $\mathbf{x}_t$ in each MuENLTree, $\mathcal{D}_t(\mathbf{x}_t) = 1$, i.e., having a new label if $\mathbf{x}_t$ falls outside the ball. Otherwise, $\mathcal{D}_t(\mathbf{x}_t) = -1$, i.e., $\mathbf{x}_t$ has no new

4. In the case that $|S| = 1$, the center and the radius of its parent node are used instead.

labels. The final output of MuENLForest is decided via majority voting.

The key idea is explained as follows. Recall that instances within the same leaf node must be similar on some attributes of either features or predictive values, or both. As a result, if a test instance falls into a certain leaf node but outside the ball. This suggests that this instance must be similar on some of the attributes to the other instances within that leaf node, but it is very different on the other attributes (features or predictive values). Thus, this instance may hold a new label with high probability.

3.2.5 Multi-Label Classifier Update

When buffer B is full, $\mathcal{H}_t$ is to be updated. This update includes the construction of a new classifier for the new label, and the update of the existing multi-label classifier. Because the detector is not perfect, it may miss some positive instances with the new label or mistake some negative ones to be positive. We aim to produce a robust classifier which can tolerate this kind of errors to some extent.

The basic idea is to introduce a latent variable which estimates the true label assignment of each instance in $\mathcal{X}_t$, where a predicted label by the detector is the initial value of the latent variable. Then the optimization learns this latent label assignment and the classifier simultaneously that best fits the data. In this way, the learned classifier is more tolerant to the errors of the detector.

ptLet X_B be the instances collected in the buffer and X_U be the set of instances with (predicted) known labels only, where $X_U = X_t \setminus X_B$. Let $\mathbf{d} = [d_1, d_2, \dots, d_m]^\top$ be the potential assignment of the new label of $X_t = [X_B; X_U]$, where m is the number of instances in $[X_B; X_U]$; and $d_k = 1$ if $\mathbf{x}_k \in [X_B; X_U]$ holds a new label; $d_k = 0$ otherwise. Note that $\mathbf{d}$ is initialized with the labels predicted by the detector.

In order to obtain a robust classifier, we propose Multi-label learning with New Labels (MNL) to simultaneously learn the $\mathbf{d}$ and classifier $\mathbf{w}_a$ for the new label $\ell \leftarrow \ell + 1$. This is done by modifying Eq. (2) in Section 3.2.1 i.e., replacing $y_{i,k}$ with $2d_k - 1$, that optimizes both the pairwise label ranking loss (the first term, to encourage positive labels to be ranked before negative labels), the misclassification loss (the second term, to encourage that labels are correctly predicted), and the regularizers (the last two terms). The optimization problem of building classifier $\mathbf{w}_a$ and learning $\mathbf{d}$ for the new label ℓ is cast as follows:

$$\begin{aligned} \min_{\mathbf{w}_a, \mathbf{d}} \quad & \sum_{j=1}^{\ell} \sum_{k=1}^m [1 - (2d_k - 1 - y_{j,k})(f_{\ell,k} - f_{j,k})]_+ \\ & + \lambda_1 \sum_{k=1}^m [1 - (2d_k - 1)f_{\ell,k}]_+ + \frac{\lambda_2}{2} \|\mathbf{w}_a\|^2 + \frac{\lambda_3}{2} \|\mathbf{d}\|^2, \\ \text{s.t.} \quad & d_k \in \{0, 1\}, k \in \{1, 2, \dots, m\}, \end{aligned} \quad (3)$$

where $y_{j,k} = h_{t,j}(\mathbf{x}_k)$ and $\lambda_1, \lambda_2, \lambda_3$ are three parameters.

The above optimization problem is a NP-Hard problem. To simplify the problem, we relax the constraint from $d_k \in \{0, 1\}$ to $d_k \in [0, 1]$. The process optimizes $\mathbf{d}$ and $\mathbf{w}_a$ alternatively.

Specifically, when fixing $\mathbf{w}_a$ and updating $\mathbf{d}$, it solves the following subproblem:

$$\begin{aligned} \min_{\mathbf{d}} \quad & \sum_{j=1}^{\ell} \sum_{k=1}^m [1 - (2d_k - 1 - y_{j,k})(f_{\ell,k} - f_{j,k})]_+ \\ & + \lambda_1 \sum_{k=1}^m [1 - (2d_k - 1)f_{\ell,k}]_+ + \frac{\lambda_3}{2} \|\mathbf{d}\|^2, \\ \text{s.t.} \quad & d_k \in [0, 1], k \in \{1, 2, \dots, m\}. \end{aligned} \quad (4)$$

Then, when updating $\mathbf{w}_a$ with $\mathbf{d}$ fixed, it solves the following subproblem:

$$\begin{aligned} \min_{\mathbf{w}_a} \quad & \sum_{j=1}^{\ell} \sum_{k=1}^m [1 - (2d_k - 1 - y_{j,k})(f_{\ell,k} - f_{j,k})]_+ \\ & + \lambda_1 \sum_{k=1}^m [1 - (2d_k - 1)f_{\ell,k}]_+ + \frac{\lambda_2}{2} \|\mathbf{w}_a\|^2, \end{aligned} \quad (5)$$

To solve Eqs. (4) and (5), we calculate the subgradient of the objective function and perform the gradient descent. After each update, we project $\mathbf{d}$ to $[0, 1]$: $\mathbf{d} \leftarrow \min(1, [\mathbf{d}]_+)$, to satisfy the box constraint.

In order to achieve a faster convergence and a good result, we adopt a warm start strategy. For the first iteration, we employ X_B as positive instances and $[X_0; X_U]$ as negative instances to train a linear multi-label classifier. For each subsequent iteration, the optimization begins with the optimization result obtained in the last iteration.

Procedure 2 summarizes the procedure of building a classifier for MNL.

When an instance with a new label appears, the influence of the new label shall be taken into consideration in updating the existing classifier. To achieve this goal, we slightly adjust each existing classifier $\mathbf{w}_i, i \in \{1, \dots, \ell - 1\}$ via minimizing the ranking loss, and penalizing a large change of classifier $\mathbf{w}_i$ as follows:

$$\min_{\mathbf{w}'_i} \sum_{j=1}^{\ell} \sum_{k=1}^n [1 - (y_{i,k} - y_{j,k})(f'_{i,k} - f_{j,k})]_+ + \|\mathbf{w}'_i - \mathbf{w}_i\|_2^2.$$

When $j = l_{mx}$, we use $\mathbf{d}$ as the new label vector. Note that this optimization does not affect the order of predicted values on all labels for each instance, since the relative ranking between any two known labels is not changed. Besides, a large change in the classifier is prevented via minimizing $\|\mathbf{w}'_i - \mathbf{w}_i\|_2^2$, since we wish to slightly adjust the existing classifier and avoid a sudden change. In a nutshell, the formulation considers the influence of new class model and it is also robust to errors made by the new label classifier to update known label classifiers.

3.2.6 Detector Update

To update the detector, we simply rebuild the detector based on X_t as follows: Each of g MuENLTrees is built using ψ instances selected from X_t based on sampling weight s_t . This produces $\mathcal{D}_{t+1}$ to replace $\mathcal{D}_t$.

4 THE MUENLHD APPROACH

In some real applications, streaming data are high-dimensional and in sparse representations. Micro-blogs for example [28], if bag-of-words features are applied, there will be over 10,000 features and each instance has sparse representation because a few out of the over 10,000 features are related to a specific topic.

Procedure 2. MNL

Input: $X_B, X_U, w_i, i \in \{1, 2, \dots, \ell\}, d_{init}, w_{a,init}$
Output: w_a

- 1: Initialize $d_0 \leftarrow d_{init}; w_{a,0} \leftarrow w_{a,init};$
 - 2: $t \leftarrow 1;$
 - 3: **repeat:**
 - 4: $d_t \leftarrow$ solve Eq. (4) with a warm start using $d_{t-1};$
 - 5: $w_{a,t} \leftarrow$ solve Eq. (5) with a warm start using $w_{a,t-1};$
 - 6: $t \leftarrow t + 1;$
 - 7: **until** converge or the maximum number of iterations is reached;
 - 8: $w_a = w_{a,t}.$
-

When we need to detect the appearance of a new topic from a sparse high-dimensional data stream, the previous tree-based detector is not suitable. Recall that in MuENLTree, we randomly select some attributes (including features and predicted values) to split the node into two son nodes. However, when the data is high dimensional and sparse, attributes with 0 value are selected with high probability. As a result, even instances belonging to different topics may be deemed similar because they both hold 0 on the selected attributes. Thus the detection accuracy is poor. Neither the use of non-zero attributes only nor building a tree which includes all possible attribute combinations is a feasible solution. Globally, most attributes have non-zero values in at least some instances in the training set, even though individual instances have few non-zero attributes. As the number of attribute combinations is exponentially large w.r.t. the number of attributes, both the time complexity and tree size are prohibitive in practice.

A practical way to handle the novel label detection problem in sparse high-dimensional data is to do a dimension reduction first. Then, the MuENL approach can be applied to the data with reduced dimensions.

Streaming kernel principal component analysis approach or SKPCA [29] is a state-of-the-art non-linear dimension reduction approach on data streams, with high efficiency and strong theoretical guarantee. Specifically, it first generates random Fourier features which are equivalent to non-linear kernel mappings. Then, a frequent direction approach is applied in order to keep the most informative direction on the spectral space via a truncated singular value decomposition (SVD), i.e., only half of the largest eigen-values are kept.

Note that SKPCA is for dimension reduction only. In this work, we incorporate SKPCA into MuENL to handle high dimensional data stream with emerging new labels. SKPCA consists of two components:

- The random feature map (Procedure 3) is defined as: $\phi = [\phi_1, \dots, \phi_m]$, where $\phi_i(a) = \cos(r_i a + \gamma_i), i \in \{1, 2, \dots, m\}, r_i \sim \mathcal{N}(0, 2sI), \gamma_i \sim \mathcal{U}(0, 2\pi), s$ is a parameter⁵ and I is the identity matrix. With this random feature mapping, sparse high dimensional sparse data is transformed into dense m dimensional representations. However, the size m of ϕ is usually chosen to be hundreds or thousands in order to achieve a good performance. $m = 2000$ and $s = 1$ are fixed in our experiments.

- The Frequent Direction (FD) update rule performs further dimension reduction in a data stream (Procedure 4), where it converts the m dimensional space to a d' dimensional space. The lower dimension d' is usually set much smaller than m . We use $d' = 100$ in the experiments.

Procedure 3. RandomFeatureMap

Input: s, m
Output: mapping function $\phi = [\phi_1, \dots, \phi_m]$

- 1: **for each** $i \in \{1, \dots, m\}$
 - 2: sample $r_i \sim \mathcal{N}(0, 2sI)$, where $\mathcal{N}$ represents normal distribution ;
 - 3: sample $\gamma_i \sim \mathcal{U}(0, 2\pi)$, where $\mathcal{U}$ represents uniform distribution;
 - 4: $\phi_i = \cos(r_i a + \gamma_i)$, where a is the input variable for ϕ ;
 - 5: **end**
-

Procedure 4. FDUpdate

Input: Matrix P , low dimensionality d'
Output: P and W

- 1: $[\cdot, \Sigma, W] = \text{SVD}(P)$, where $\Sigma = \text{diag}(\{\sigma_i | i \leq d'\})$;⁶
 - 2: $\text{mid} = \text{Round}(d'/2);$
 - 3: $\tau = \Sigma_{\text{mid}, \text{mid}};$
 - 4: $P \leftarrow \sqrt{\max(0, \Sigma^2 - \tau^2)} W.$
-

Incorporating these two components with MuENL, we have MuENLHD summarized in Algorithm 2. Specifically, the output of RandomFeatureMap is a set of mapping functions, each is a random Frontier transformation. This is produced before the beginning of the stream. It needs to be done once only; and the mapping functions are used for the rest of the stream.

The output of FDUpdate is an updated matrix P and a linear mapping W . According to SKPCA [29], P is a good spectral approximation of the data stream with a bounded error, and W is the corresponding mapping. In other words, P gradually captures the main information of the data stream in the spectral space.

The detector and the classifier in MuENLHD are built with the new mapped space, from the output of FDUpdate. In order to reduce the complexity, we only update the mapping when buffer B is full. The classifier and the detector are updated using B .

5 TIME COMPLEXITY

In this section, the time complexity of each component of MuENL and MuENLHD is presented. The common components of MuENL and MuENLHD are the multi-label classifier (PLR), the detector MuENLForest, and update the classifier (MNL) and the detector (which is equivalent to building a detector). For MuENLHD, there are two additional components RandomFeatureMap and FDUpdate.

For the multi-label classifier (PLR) and update (MNL), a pairwise label ranking optimization is involved which has time complexity $O(|v|^2 nd)$, where $|v|$ is the size of the class

5. Note that s is equivalent to the parameter in a rbf kernel because $\phi(a_i)^T \phi(a_j)$ is a good approximate to Gaussian kernel $K(a_i, a_j) = \exp(-s \|a_i - a_j\|^2)$.

6. The number of rows of P should be larger or equal to d' . We assume that singular value decomposition SVD returns singular values in a descending order (σ_i denotes the i -th largest singular value).

TABLE 1
Time Complexity of Each Component of MuENL and MuENLHD

	PLR	MNL	MuENLForest	RandomFeatureMap	FDUpdate
MuENL	$O(v ^2 nd)$	$O(v ^2 nd)$	$O(q g_e m \psi)$	—	—
MuENLHD	$O(v ^2 nd')$	$O(v ^2 nd')$	$O(q g_e m \psi)$	$O(mdn)$	$O(d'mn)$

label set, n is the number of instances, and d is the number of dimensions. For the detector construction, each node involves k-means clustering with 2 clusters which have time complexity $O(|q|n)$. The MuENLTree construction involves ψ instances having tree height limit e_m . Thus, the time complexity is $O(|q|e_m \psi)$. As a result, the time complexity of MuENLForest with g MuENLTrees is $O(|q|g_e m \psi)$. The detector update simply rebuilds the detector, which will be $O(|q|e_m \psi)$. RandomFeatureMap and FDUpdate have $O(mdn)$ and $O(d'mn)$ respectively, where m is the number of random features, and d' is the number of features at the output of FDUpdate. The time complexity of each component is summarized in Table 1.

Algorithm 2. MuENLHD

Input: $X_0, Y_0, \{x_t, t \in \{1, 2, \dots, T\}\}$, low dimension d'

Output: $\hat{y}_t$ for each x_t

```

1: Generate a random feature map
    $\phi = \text{RandomFeatureMap}(s, m)$ ;
2: Initialize  $P, W : [P, W] = \text{FDUpdate}(\sqrt{\frac{2}{m}}\phi(X_0), d')$ ;
3: Initialize sampling weight vector  $s_0 = \mathbf{1}_{|X_0|}$ ;
4: Initialize  $B = \emptyset$ ;
5: Train  $\mathcal{H}_0$  with  $Z_0 = \sqrt{\frac{2}{m}}\phi(X_0)W$  and  $Y_0$ ;
   build detector  $\mathcal{D}_0$  on  $Z_0$ ;
6:  $\mathcal{H}_1 = [\mathcal{H}_0, \mathcal{D}_0]$ ;  $\mathcal{D}_1 = \mathcal{D}_0$ ;  $W_p = W$ ;  $t = 1$ ;
7: repeat
8:   Receive a new instance  $x_t$ , obtain its low dimensional
      representation  $z_t = \sqrt{\frac{2}{m}}\phi(x_t)W$ ;  $X_t = [X_{t-1}; x_t^T]$ ;
9:   Update sampling weight vector  $s_t = [s_{t-1}; 1]$ ;
10:  Insert  $z_t$  into zero valued rows in  $P$ ;
11:  if  $P$  has no rows having zero value.
12:     $[P, W_p] = \text{FDUpdate}(P, d')$ ;
13:  end if
14:   $\hat{y}_t = \mathcal{H}_t(z_t)$ , where  $\mathcal{H}_t = [h_{t,1}, h_{t,2}, \dots, h_{t,|v_t|}, \mathcal{D}_t]$ ;
15:  if  $\mathcal{D}_t(z_t) = 1$ 
16:    Add  $z_t$  to  $B$ ;
17:    if  $|B| = \text{MAX\_BUFFER\_SIZE}$ 
18:       $W \leftarrow W_p$ ;  $Z_t = \sqrt{\frac{2}{m}}\phi(X_t)W$ ;
19:      Update  $\mathcal{D}_{t+1}$  and  $\mathcal{H}_{t+1}$  using  $\mathcal{T}_t = (Z_t, \mathcal{H}_t(Z_t))$ ;
20:      Empty  $B$ ;
21:      The new label is converted to the known label:
       $\ell \leftarrow \ell + 1$ ;  $v_t = v_{t-1} \cup \{\ell\}$ ;
22:      Update sampling weight vector  $s_t \leftarrow 0.8s_t$ ;
23:      Update  $X_t$  and  $s_t$  via keep or discard instances
      according to  $s_t$ ;
24:    end if
25:  end if
26:   $t \leftarrow t + 1$ ;
27:   $v_t = v_{t-1}$ ;  $\mathcal{D}_t = \mathcal{D}_{t-1}$ ;  $\mathcal{H}_t = \mathcal{H}_{t-1}$ ;
28: until  $t = T$ .
```

Note that, even though there are two additional components for MuENLHD, MuENLHD is faster than MuENL when handling high dimensional data in practice. The reasons

are: (a) In MuENLHD, much lower dimensional data are involved in PLR, MuENLForest and MNL, which will contribute to much shorter running time than MuENL; (b) the time complexities of RandomFeature and FD are much smaller than those of PLR and MNL; (c) with a much smaller number of dimensions, the best $|q|$ selected for MuENLForest in MuENLHD is much smaller than that for MuENL.

6 EXPERIMENTS ON LOW DIMENSIONAL DATA

6.1 Configuration

To evaluate the predictive performance of the MuENL approach, we use 5 multi-label benchmark datasets (“Birds”, “CAL500”, “Emotions”, “Enron” and “Yeast”, detailed information is shown in Table 2).⁷ A data stream is simulated by generating the initial set of labeled data (i.e., (X_0, Y_0)) and individual unlabeled instances x_t arrive progressively in time step $t \in \{1, 2, \dots, T\}$.

Fig. 1 shows an example simulation of a data stream using the Yeast dataset, where 5 new labels (A to E) are observed in different time periods. At t_0 , only the initial training set with known labels (v_0) is observed; and a multi-label classifier is trained using this training set. Then, instances with possibly new label A begins to appear in the $t_0 - t_2$ period. At t_1 , which denotes that the buffer (of instances having detected with the new label) is full, MuENL updates the multi-label classifier ($[h_{t,1}, \dots, h_{t,\ell}]$) for the expanded known label set $\{v_0, A\}$; and also updates the detector ($\mathcal{D}_t$) so that it can detect the next new label.

The updated classifier and detector are then used for prediction at $t_1 + 1$, which is the time that A becomes part of the known labels. At t_2 , new label B begins to appear, and the same process in the $t_0 - t_2$ period repeats in the $t_2 - t_4$ period; and the subsequent periods up to $t_8 - t_{10}$ for label E. The classifier and detector used for prediction in each period are summarized in Table 3. Note that for detection, we always use MuENLForest; for prediction, when the classifier has not been built for the new label, we use the output of the detector MuENLForest as the prediction result; after MNL has been updated for the new label, we use MNL for prediction.

It is interesting to note that the overall predictive performance does not degenerate much with the successive appearance of new labels, as shown in Fig. 1, after about 900 time steps with 5 consecutive new labels.

For a given dataset, the above simulation is conducted as follows. The label set v in a given dataset is split into two subsets, i.e., $v_N = \{v_{a1}, \dots, v_{a5}\}$ the candidate new label set of five new labels, and v_K the known label set. Let P_K be the data subset without v_N ; and P_N be the data subset with v_N . Note that we need to do some pre-processing of the given dataset: (1) if two labels are highly correlated, i.e., often co-occur, they are combined by taking a union of the two labels, since it is impossible to distinguish these labels

7. <http://mulan.sourceforge.net/datasets.html>

TABLE 2
Characteristics of Datasets Used

Dataset	# inst.	# dim.	#labels	# cardinality
Birds	645	260	19	1.014
CAL500	502	68	174	26.044
Emotions	593	72	6	3.378
Enron	1702	1001	53	3.378
Yeast	2417	103	14	4.237
Weibo	6000	15461	10	1.593

inst is the number of instance, # dim is the dimension, #label is the total number of class labels, and #cardinality is the average number of labels for each instance

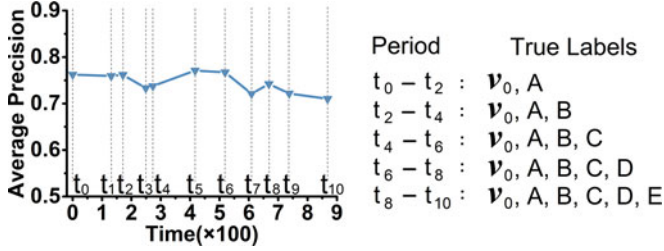


Fig. 1. Performance on Yeast dataset. Five new labels are involved.

without any prior knowledge. (2) If a label is independent of other labels, i.e., seldom co-occur with other labels, it is not as a new label, since this kind of label will degenerate to a new class detection in class incremental learning, which is an easier task; (3) those moderately correlated labels (around median in the ranked list) are potential new labels. These labels sometimes co-occur with the same label and sometimes with different labels.⁸

We sample 90 percent of P_K to form the initial training set $T_0 = (X_0, Y_0)$. For $t > 0$, x_t is randomly selected uniformly (without replacement) from $P_K \setminus T_0$ and a subset of P_N having label v_{ai} for time period $t_{2i-2} - t_{2i}$ where $i = 1, \dots, 5$.

For evaluation, we consider a commonly applied metric for multi-label learning, i.e., Average Precision. It is the average fraction of positive labels ranked higher than a particular positive label. Let p denote the number of test instances; C_i^+ and C_i^- be the sets of positive and negative labels, respectively, associated with instance x_i . The set of positive predicted labels which are ranked lower than label c for x_i is defined as: $\hat{Q}_{i,c} = \{j \mid \text{rank}(x_i, j) \leq \text{rank}(x_i, c), j \in C_i^+\}$, where $\text{rank}(x_i, c)$ is the rank of label c in the predicted label ranking (sorted in descending order). Then, Average Precision = $\frac{1}{p} \sum_{i=1}^p \frac{1}{|C_i^+|} \sum_{c \in C_i^+} \frac{|\hat{Q}_{i,c}|}{\text{rank}(x_i, c)}$. The larger the Average Precision, the better.

The experiment is conducted based on the simulation described above. The predictive performance is measured at $t_1, \dots, t_6$; and two kinds of evaluations are conducted: (i) v_t -evaluation: it measures the performance with the emergence of new labels in the simulation; (ii) v_0 -evaluation: it assesses how well MuENL performs on the initial label set v_0 only throughout the entire period, i.e., the performance on the emerging new labels are not assessed.

8. The correlation between two labels is measured based on cosine similarity. In order to rank label correlation among labels, we use the sum of cosine similarities of pair-wise labels. A, B, C three label vectors for example, the score for A is $\cos(A, B) + \cos(A, C)$.

TABLE 3
Models Used for Predicting New Labels in the Simulated Time Periods

Time	Model for predicting new labels	True labels
$t_0 - t_1$	MuENLForest	v_0, A
$t_1 - t_2$	MNL	v_0, A
$t_2 - t_3$	MuENLForest	v_0, A, B
$t_3 - t_4$	MNL	v_0, A, B
$t_4 - t_5$	MuENLForest	v_0, A, B, C
$t_5 - t_6$	MNL	v_0, A, B, C
...	...	...

For each dataset, the average result and standard deviation of 10 independent runs of simulations are reported.

The performances of two algorithms are said to have a significant difference if the difference is more than two standard errors.

6.2 Baselines

We employ two sets of baselines to compare with MuENL: (i) the state-of-the-art multi-label approaches; (ii) variants of MuENL having different MuENL components.

6.2.1 State-of-the-Art Multi-Label Approaches

We compare MuENL with BR [1], CLR [7], ECC [8], PLR, LIMO [30], and GenEML [23], which are multi-label learning approaches considering the initially known labels only. BR, CLR, and ECC are first-order, second-order and high-order multi-label learning approaches, respectively. PLR is the multi-label approach proposed in Section 3.2.1. It is a degenerated version of MuENL without a detector and model update. LIMO and GenEML are the most recently proposed approaches to handle multi-label learning: LIMO considers both the instance-wise and label-wise margin and GenEML is a generative model. Further details are listed as follows:

- BR trains a linear classifier for each label independently.
- CLR transforms the multi-label learning problem into the label ranking problem and incorporates a virtual label to separate the relevant and irrelevant labels.
- ECC is the ensemble of classifier chains. In each chain, ground truth labels are encoded into the feature space gradually; thus high order label relations are exploited.
- PLR takes advantage of pairwise label ranking.
- LIMO maximizes both the label-wise margin and instance-wise margin.
- GenEML is a flexible and scalable generative model based on a latent factor model for the label matrix.

6.2.2 Variants of MuENL

To further validate the effectiveness of each component, we compare MuENL with its variants.

- MuENL_{IF}: Use iForest [21] (instead of MuENLForest) as the detector.
- MuENL_{OC}: Use OC-SVM [20] (instead of MuENLForest) as the detector.
- MuENL_{SVM}: Use MuENLForest as the detector, but use a different classifier: train a linear classifier for

TABLE 4
MuENL Variants

Approach	Classifier	Detector	Classifier Update
MuENL _{SVM}	PLR	MuENLForest	SVM
MuENL _{IF}	PLR	iForest	MNL
MuENL _{OC}	PLR	OC-SVM	MNL
MuENL _{OR}	PLR	MuENLForest	Oracle+PLR
MuENL	PLR	MuENLForest	MNL

the new label via SVM by using the same training set as used in MNL.

- MuENL_{OR}: Use MuENLForest as the detector and assume that an oracle is accessible to provide the ground truth for model updates. Its performance will be the upper bound.

These variants are summarized in Table 4.

The following codes are used to implement MuENL: LIB-LINEAR toolbox [31] is applied as the linear SVM; MANOPT toolbox [32] is utilized to implement the steepest descent with a line search procedure to update the multi-label classifier. kmeans with $k = 2$ is applied to implement the splitting criterion in MuENLTree which is a binary tree.

For each method under comparison, its parameters are tuned using the initial training set at t_0 via 5-fold cross-validation. Then, these settings are employed for the rest of the data stream.

To select the appropriate settings for the detector, a label randomly selected from Y_0 is used for validation, and the rest of the labels are used to train a detector. This process is repeated 5 times for a different randomly selected label in Y_0 in order to choose the appropriate parameter settings for the detector. The ranges of values used are: buffer size $|B| \in \{10, 15, 30, 60\}$, maximum tree height $e_m \in \{-2, -1, 0, 1\} + \text{ceil}(\log_2 \psi)$ (where $\text{ceil}(\log_2 \psi)$ is the average height of the binary tree with sample size ψ), number of selected attributes $|q| \in \{1, 3, 5\}$. The other parameters are fixed by default: tree number $g = 100$, sample size $\psi = 256$ and $\lambda_3 = 1$.

Detail ranges (or values) of the parameters of MuENL and the baselines are summarized in Table 5.

6.3 Experimental Results

We conduct two sets of experiments which differ in the number of new labels which may appear in each time period. The first set has exactly one new label; whereas the second set has more than one new label. The second set assesses how multiple new labels in each time period may affect the predictive performance.

6.3.1 Results on One New Label per Time Period

Compare with Existing Multi-label Approaches. Fig. 1 is an example plot of v_t -evaluation on the Yeast dataset having one new label in each time period over 5 periods.⁹ In general, MuENL maintains a good predictive performance spanning the entire duration from t_0 to t_{10} . This is a direct result of a good detector and a robust classifier in MuENL.

9. Notice that the time when the buffer gets full may be different in different time periods, and for different detectors in the same time period. Thus, the duration of one period may vary from one period to another and from one detector to another.

TABLE 5
Parameters Used in MuENL and Its Baselines

Parameter	Model	Approach
$ B \in \{10, 15, 30, 60\}$	—	MuENL MuENL variants
$\lambda_1, \lambda_2 \in \{0.001, 0.01, 0.1, 1\}$	PLR	MuENL MuENL _{IF} MuENL _{OC} MuENL _{OR} PLR & MNL
$ q \in \{1, 3, 5\}$ $g = 100$ $\psi = 256$ $e_m \in \{6, 7, 8, 9\}$	MuENLForest	MuENL MuENL _{SVM}
$\lambda_3 = 1$	MNL	MuENL MuENL _{OC} MuENL _{IF}
$C \in \{0.1, 1, 10, 100\}$	SVM	MuENL _{SVM} BR, CLR, ECC
$g = 100$ $\psi = 256$ $e_m \in \{6, 7, 8, 9\}$	iForest	MuENL _{IF}
$nu \in \{0.3, 0.5, 0.7\}$	OC-SVM	MuENL _{OC}

Fig. 2 summarizes the comparison with BR, ECC, CLR, PLR, LIMO, and GenEML in terms of v_t -evaluation in five datasets. It is interesting to note that MuENL almost always performs better than all baselines. Many of the differences are significant.

The summarised result on five datasets for v_0 -evaluation is provided in Fig. 3 in terms of average precision. MuENL achieves better or comparable performance in comparison with all baselines. This shows that MuENL, which incorporates detection and prediction of new labels, not only does no harm to the performance on known labels, but can also gain better performance because pairwise label ranking is considered.

Compare with MuENL Variants. Fig. 4 presents the result based on average precision.

Among the four MuENL variants, MuENL_{OR} has the best performance since it uses the oracle which provides the ground truth that is not available to the other variants. Note that this oracle is available in practice and it is used to show the upper bound.

MuENL obtains a performance comparable with MuENL_{OR} in all cases, where there is no significant difference in performance. This is a direct result of a good detector and a robust classifier for both known and new labels.

MuENL achieves a significantly better performance than MuENL_{SVM} on all datasets. This shows that the robust classifier update procedure in MuENL works better than that in SVM; and it is essential in maintaining a good performance in a dynamic learning environment.

MuENL_{IF} and MuENL_{OC} replace the detector in MuENL with iForest and OC-SVM, respectively. Even though a robust update procedure is applied in both of them, MuENL still performs better than them (on 4 of 5 datasets, MuENL is significantly better). This validates the effectiveness of the detector in MuENL. Because new labels often co-occur with known labels, this kind of occurrences confuses existing detectors OC-SVM or iForest—leading to low detection accuracy. By taking both the feature and label information

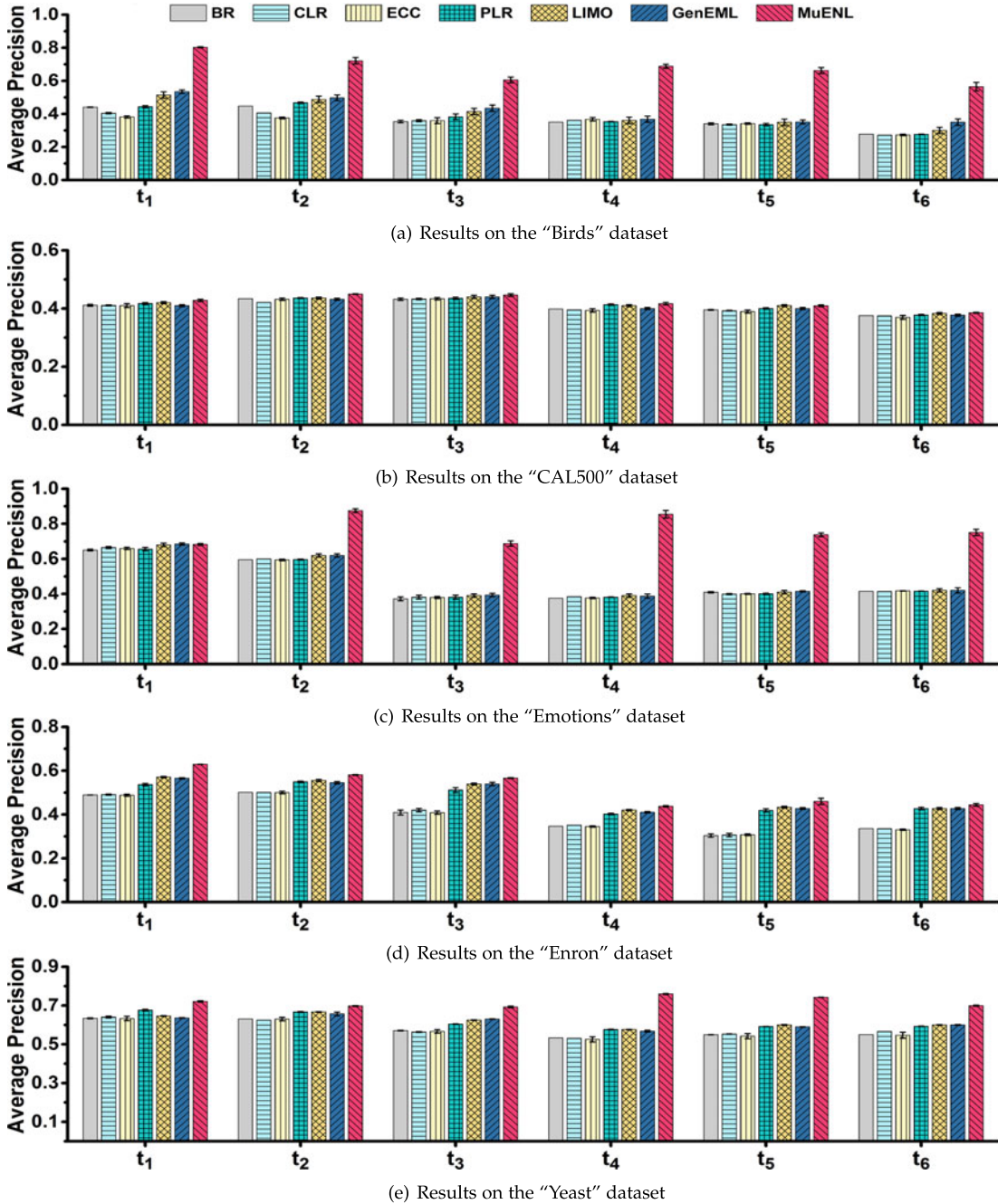


Fig. 2. Compare BR, CLR, ECC, PLR, and MuENL: Results using v_t -evaluation. $t_1 - t_6$ on the horizontal axis are the six time points.

into account, MuENLForest is able to differentiate instances with a combination of features and label patterns due to new labels from that due to existing labels. This yields a better detection outcome as a result.

A more direct analysis of the above comparison is provided in Fig. 5 which shows detection performance of MuENLForest, OC-SVM and iForest in terms of the F1-score evaluated at t_1 . As expected, MuENLForest outperforms the other competitors.

For the classifier of new labels, we make a more direct comparison of the proposed MNL with SVM and PLR which treat the instances in the buffer as positive ones and other seen instances as negative ones in order to train a classifier. Fig. 6 shows the performance evaluated at t_2 (In this experiment, the same detector MuENLForest is applied). Fig. 6

shows that MNL is better than SVM and PLR because it employs a robust update, which works well even under the condition that a detector is imperfect.

To examine the significance of the relative performance among these algorithms, we perform a post-hoc Nemenyi test [33]. The result shown in Fig. 7 reveals that the proposed MuENL is significantly better than all the traditional multi-label approaches (which do not consider any new labels) and other MuENL variants, except MuENL_{OR} in which the oracle is available.

Compare with LP-SENCForest. SENCForest [34] is proposed recently to handle class incremental learning on streaming data, and has achieved a success. It provides a unified tree-based model for new class detection and known class prediction. SENCForest focuses on a multi-class

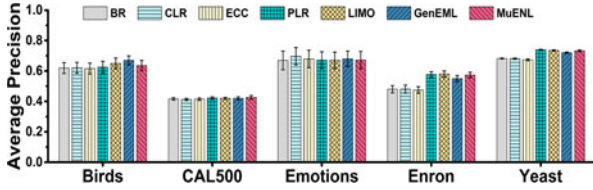


Fig. 3. Compare BR, CLR, ECC, PLR, LIMO, GenEML, and MuENL: Results using v_0 -evaluation.

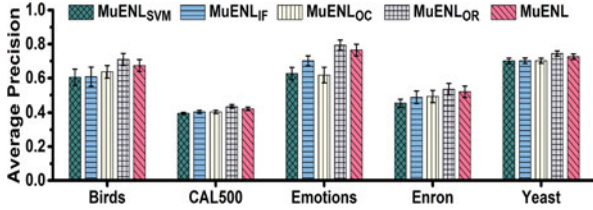


Fig. 4. Compare MuENL variants: Results using v_t -evaluation.

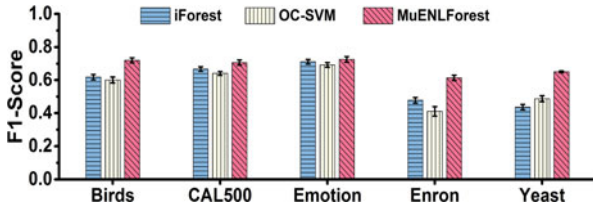


Fig. 5. Detection performance of MuENLForest, OC-SVM, and iForest (i.e., detectors in three MuENL variants) at t_1 .

setting—each instance belongs to a single class only. To apply SENCForest in the multi-label learning setting, we first apply Label Powerset (LP) [35] to encode different label combinations as different classes to transform multi-label learning into multi-class setting, before SENCForest is applied. We name it LP-SENCForest, which is the counterpart of SENCForest in our setting.

Fig. 8 summarizes the comparison results of averaged performance on the whole data stream in five datasets. As can be observed, MuENL is much better than LP-SENCForest. This result is not surprising because: (1) LP transformation leads to too many classes, some of them hold a small number of instances. SENCForest does poorly because it requires a sufficient number of instances for each class in order to do well. (2) After LP transformation, the detected new class by SENCForest can be a new combination of existing labels, but does not contain any new labels.

Results with Different Label Sequences. In order to further validate the effectiveness of our approach, the evaluation is conducted multiple trials on a dataset, where each trial has a different random order of known and new labels on the simulated stream. The comparison is done with two best performing methods in each of two approaches, i.e., the multi-label approach: PLR and LIMO,¹⁰ and twoMuENL variants: MuENL_{OC} and MuENL_{IF}.

We report the averaged difference from the baseline BR in MicroF1, which is a label based measure: $\text{MicroF1} = \frac{2 \sum_{i,j} y_{i,j} h_{i,j}}{\sum_{i,j} y_{i,j} + \sum_{i,j} h_{i,j}}$, $i \in \{1, \dots, T\}$, $j \in \{1, \dots, |v_t|\}$, on the entire data stream as in Fig. 9. The result that MuENL outperforms these four methods, even under random orders of new labels.

10. GenEML is also comparable, not listed here due to the space issue.

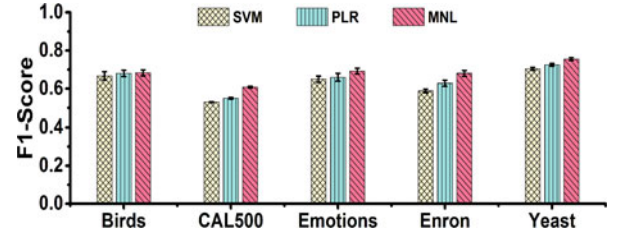


Fig. 6. Classifier performance of MNL, SVM, and PLR (i.e., classifiers in three MuENL variants) at t_2 .

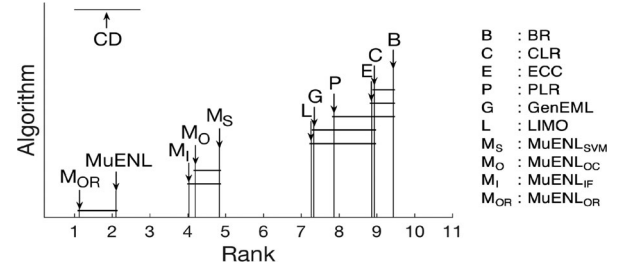


Fig. 7. Critical difference (CD) diagram of the post-hoc Nemenyi test ($\alpha = 0.05$) for the comparison results with both the traditional multi-label learning approaches and MuENL variants. The difference between two algorithms is significant if the gap between their ranks is larger than the CD. There is a line between two algorithms if the rank gap between them is smaller than the CD.

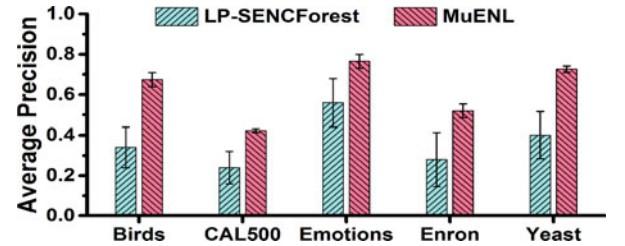


Fig. 8. MuENL versus LP-SENCForest.

6.3.2 Results on Multiple New Labels per Time Period

When more than one new label occurs in a time period, MuENL can also be used by treating these new labels as a single meta label. Specifically, if any instance has a subset of labels, encapsulated by a meta-label, then the instance has this meta-label. The goal in this setting is to detect and classify a new meta label, if it exists, in each time period.

We conduct experiments, where the number of new labels varies from 1 to 9 in each time period, to examine the effect of multiple labels in comparison with the baseline which has only one new label per time period. The evaluation is based on the meta label we specified at the beginning of the data stream.

Fig. 10 shows that results in two datasets. Both results show that the MuENL approach can handle multiple new labels; and in many cases, it performs better in the multiple-new-labels scenarios than in the single-new-label scenarios. This is because the learning task for a meta label of multiple labels can be easier than that for a single label, depending on the data distribution. An example is shown Fig. 10a.

7 EXPERIMENTS ON HIGH DIMENSIONAL DATA

7.1 Dataset

We have collected a total of 6,000 Weibo instances in 10 topics: traffic, car, president, Nobel prize, TV show, music, topics: traffic, car, president, Nobel prize, TV show, music,

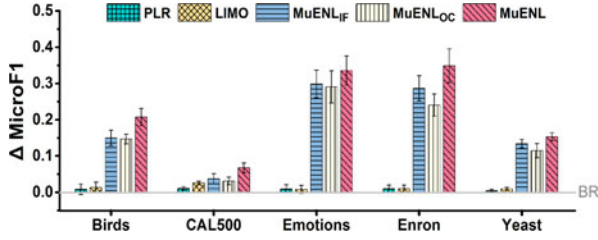


Fig. 9. Compare BR, PLR, LIMO, MuENL_{IF} , and MuENL_{OC} with different label sequences as new labels: Results using the difference in MicroF1 from BR.

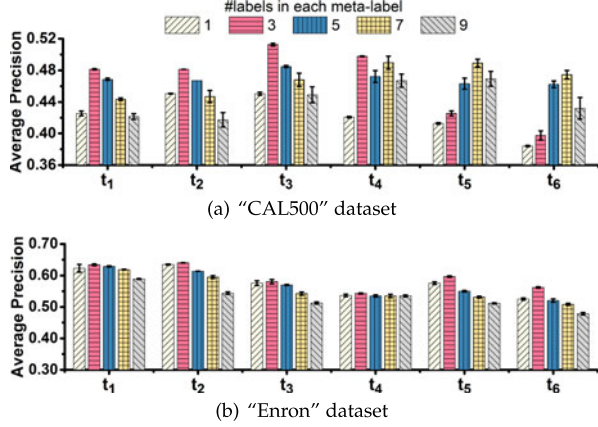


Fig. 10. MuENL 's performance in scenarios having different numbers of new labels in each time period

video, advertisement, sports, finance (indexed as 1 - 10). We extracted the bag-of-words features which resulted in 15,461 features. As a consequence, the data is very sparse, i.e., only 0.2% of the total number of features have non-zero values; and each instance belongs to 1.61 labels on average.

When it begins at t_0 , there are only five known labels, and the rest of them will successively emerge as new labels. Five new labels emerge successively at different times, denoted as $t_1, t_2, \dots, t_5$. On average, the interval between the subsequent emergence of two new labels has 1,000 instances. For experiments, we permute the order of class labels, and conduct 5 runs to avoid the influence of new label order on the results.

7.2 Baselines

We compare MuENLHD with two baselines:

- MuENL : Directly apply MuENL on the high dimensional sparse data stream.
- MuENL+PCA : First, we perform PCA on the observed data at time t_0 to obtain the mapping from the original space to a low dimensional subspace (where $d' = 100$). Then, we transform each arriving instance to the low dimensional representation via this mapping. We apply MuENL on the transformed data.
- MuENL+MultiPCA : Unlike MuENL+PCA , which performs PCA only at t_0 , MuENL+MultiPCA applies PCA multiple times, i.e., a new PCA is conducted on a data chunk collected during the period in-between the buffer is empty and full.

The parameter setting of MuENL is the same as that used in Section 6. The settings for MuENLHD are given as follows: the number of the random features $m = 2000$, and the number of attributes in the transformed space is set as $d' = 100$.

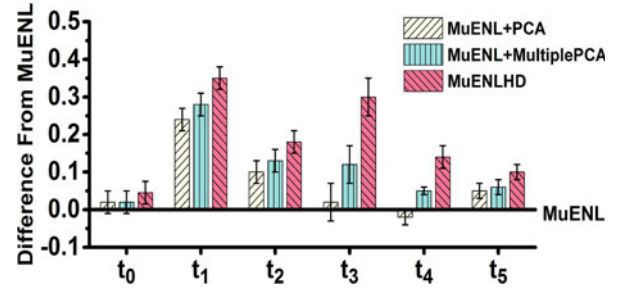


Fig. 11. Compare MuENL+PCA , MuENL+MultiPCA , and MuENLHD : Result shown is the difference from MuENL in terms of Average Precision at each time point.

7.3 Experimental Results

We evaluate the performance in terms of Average Precision on 100 instances at every time point of $t_0, t_1, \dots, t_5$. The performances of MuENLHD , MuENL+PCA , MuENL+MultiPCA are exhibited in Fig. 11. As expected, all algorithms have comparable performance at time t_0 . However, MuENLHD achieves much better performance than the other contenders at t_1 to t_5 . This is mainly due to fact that streaming kernel PCA captures the main information of the data stream in different time periods; and those transformed low dimensional representations work well collaboratively with the predictive information from the classifiers.

Although MuENL+PCA reduces the dimensionality to the same as that of MuENLHD , it performs worse because MuENL+PCA only trains PCA at t_0 , and keeps the linear mapping throughout the stream. This strategy leads to a worse performance in the following detection, since emerging new instances are not considered. In contrast, MuENLHD adapts the dimension reduction throughout the stream.

For MuENL+MultiPCA , PCA is applied each time the MuENL 's models are updated for an emerging new label. Even though PCA has been applied multiple times to adapt to the data stream, only the latest data chunk is considered. In contrast, MuENLHD adapts along the stream. As a result, MuENLHD outperforms MuENL+MultiPCA .

We also make a comparison on the time spent to handle the whole stream, shown in Table 6. As can be observed, MuENL takes nearly 100 times longer than MuENLHD to handle the high dimensional data stream. This is consistent with the factor of reduction from 15000 to 100 dimensions. Though MuENL+PCA is 3 times faster than MuENLHD , they are in the same order. MuENLHD needs the extra time in order to update the nonlinear mapping throughout the stream. MuENLHD has about the same run time as MuENL+MultiPCA , but it achieves much better performance.

8 THE EFFECT OF PARAMETERS

We study the effect of parameters in this section. The experiment is conducted in one run, where settings of the parameters are determined via 5-fold cross-validation before the start of the run. Then, a parameter is varied to study its effect while the other parameters are fixed.

Fig. 12 shows the results of the study on varying the setting of parameters (buffer size $|B|$ in Algorithm 1; λ_1, λ_2 and λ_3 in Procedure 2; e_m and $|q|$ in Procedure 1). The results show that the MuENL approach is not very sensitive to the settings of λ 's and $|q|$; but it is sensitive to $|B|$ and e_m . Its sensitivity to the buffer size is because (i) a good performing classifier cannot be trained for the new label if the buffer

TABLE 6
Total Time Used to Handle the Data Stream
with 6,000 Instances

	MuENL	MuENL+PCA	MuENL+MultiPCA	MuENLHD
Time (s)	120,839	507	1417	1,442

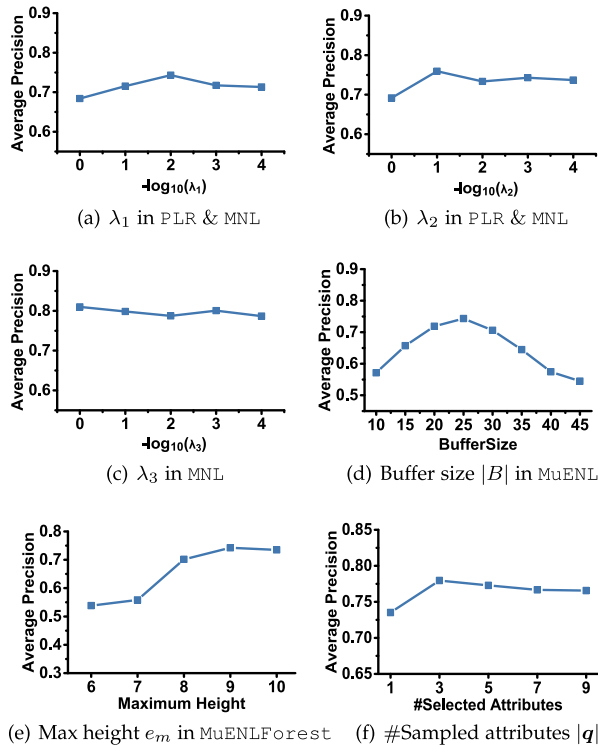


Fig. 12. Example plots of the effects of parameters in MuENL on "Emotions".

size is too small; and (ii) when the buffer size is too large, it may contain a mixture of different new labels, which will degenerate the performance.¹¹

Selecting an appropriate setting is not an issue as long as a cross-validation can be employed at the beginning of a data stream.

One issue related to memory is the expanded set of known labels as time progresses. We may exploit the "forgetting mechanism", i.e., remove models from $\mathcal{H}_t$ which have not been used for prediction (or equivalently, labels have not been seen) for a certain period of time. This mechanism will set a limit to the memory required to store the models. A recurrence of any of the removed labels will be treated as new labels.

9 CONCLUSIONS

Multi-label learning with emerging new labels is a practical problem that demands attention. This paper extends our preliminary research [36], which formalizes this problem and proposes the novel MuENL approach which has a model consists of three components: (1) a multi-label classifier for the known labels, (2) a detector for new labels, and (3) the updating processes for the classifier and the detector for each new label. Because existing methods only consider

a fixed label set, they do not have the last two components. As a result, they are significantly less effective than the proposed approach in the dynamic learning environment, as verified in the empirical evaluation. The idea of converting part of the problem into an outlier detection problem has enabled the whole problem to be solved satisfactorily. The outlier detector we have designed has high detection rate—the key to ensuring a robust classifier that can maintain a high classification accuracy in data streams. We also show that MuENL can be easily extended to handle sparse high dimensional data streams by simply reducing the original dimensionality using Streaming Kernel PCA, and then applying MuENL on the reduced dimensional space. The empirical evaluation validates its effectiveness.

ACKNOWLEDGMENTS

This research was supported by NSFC (61751306), 111 Project (B14020), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

REFERENCES

- [1] M. R. Boutell, J. Luo, X. Shen, and C. M. Brown, "Learning multi-label scene classification," *Pattern Recognit.*, vol. 37, no. 9, pp. 1757–1771, 2004.
- [2] A. McCallum, "Multi-label text classification with a mixture model trained by EM," in *Proc. Working Notes AAAI'99 Workshop Text Learn.*, 1999, pp. 1–7.
- [3] D. Turnbull, L. Barrington, D. Torres, and C. Lanckriet, "Semantic annotation and retrieval of music and sound effects," *IEEE Trans. Audio Speech Lang. Process.*, vol. 16, no. 2, pp. 467–476, Feb. 2008.
- [4] M.-L. Zhang and Z.-H. Zhou, "A review on multi-label learning algorithms," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 8, pp. 1819–1837, Aug. 2014.
- [5] Z.-H. Zhou, "Learnware: on the future of machine learning," *Frontiers Comput. Sci.*, vol. 10, no. 4, pp. 589–590, 2016.
- [6] Z.-H. Zhou, "A brief introduction to weakly supervised learning," *Nat. Sci. Rev.*, vol. 5, no. 1, pp. 44–53, 2018.
- [7] J. Fürnkranz, E. Hüllermeier, E. L. Mencía, and K. Brinker, "Multilabel classification via calibrated label ranking," *Mach. Learn.*, vol. 73, no. 2, pp. 133–153, 2008.
- [8] J. Read, B. Pfahringer, G. Holmes, and E. Frank, "Classifier chains for multi-label classification," *Mach. Learn.*, vol. 85, no. 3, pp. 333–359, 2011.
- [9] Z.-H. Zhou and Z.-Q. Chen, "Hybrid decision tree," *Knowl.-Based Syst.*, vol. 15, no. 8, pp. 515–528, 2002.
- [10] S. Rüping, "Incremental learning with support vector machines," in *Proc. 1st IEEE Int. Conf. Data Mining*, 2001, pp. 641–642.
- [11] V. Vapnik, A. Vashist, and N. Pavlovitch, "Learning using hidden information (learning with teacher)," in *Proc. Int. Joint Conf. Neural Netw.*, 2009, pp. 3188–3195.
- [12] Q. Da, Y. Yu, and Z.-H. Zhou, "Learning with augmented class by exploiting unlabeled data," in *Proc. 28th AAAI Conf. Artif. Intell.*, 2014, pp. 1760–1766.
- [13] I. Kuzborskij, F. Orabona, and B. Caputo, "From n to n+1: Multi-class transfer incremental learning," in *Proc. IEEE Conf. Comput. Vis Pattern Recognit.*, 2013, pp. 3358–3365.
- [14] X. Xu, W. Wang, and J. Wang, "A three-way incremental-learning algorithm for radar emitter identification," *Frontiers Comput. Sci.*, vol. 10, no. 4, pp. 673–688, 2016.
- [15] X. Li and Y. Guo, "Max-margin zero-shot learning for multi-class classification," in *Proc. 18th Int. Conf. Artif. Intell. Statist.*, 2015, pp. 626–634.
- [16] T. Mensink, E. Gavves, and C. G. Snoek, "Costa: Co-occurrence statistics for zero-shot classification," in *Proc. IEEE Conf. Comput. Vis Pattern Recognit.*, 2014, pp. 2441–2448.
- [17] F. Li and H. Wechsler, "Open set face recognition using transduction," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 11, pp. 1686–1697, 2005.
- [18] W. J. Scheirer, R. A. de Rezende, A. Spkota, and T. E. Boult, "Toward open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 7, pp. 1757–1772, 2013.

11. Note that the evaluation here is different from that for meta new labels shown in Section 6.3.2. For a meta new label, if any one of those labels is detected, it is a correct detection. In this setting, it is correct only if the exact label is detected.

- [19] G. Tsoumakas, I. Katakis, and L. Vlahavas, "Random k-labelsets for multilabel classification," *IEEE Trans. Knowl. Data Eng.*, vol. 23, no. 7, pp. 1079–1089, Jul. 2011.
- [20] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [21] F. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.
- [22] Y. Fu, Y. Yang, T. M. Hospedales, T. Xiang, and S. Gong, "Transductive multi-label zero-shot learning," *CoRR*, vol. abs/1503.07790, 2015, <http://arxiv.org/abs/1503.07790>.
- [23] V. Jain, N. Modhe, and P. Rai, "Scalable generative models for multi-label learning with missing labels," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 1636–1644.
- [24] M. Xu, R. Jin, and Z.-H. Zhou, "Speedup matrix completion with side information: Application to multi-label learning," in *Proc. Adv. Neural Inf. Process. Syst.* 26, 2013, pp. 2301–2309.
- [25] W. Bi and J. T. Kwok, "Multilabel classification with label correlations and missing labels," in *Proc. 28th AAAI Conf. Artif. Intell.*, 2014, pp. 1680–1686.
- [26] Y. Zhu, J. Kwok, and Z.-H. Zhou, "Multi-label learning with global and local correlation," *IEEE Trans. Knowl. Data Eng.*, to be published, doi: [10.1109/TKDE.2017.2785795](https://doi.org/10.1109/TKDE.2017.2785795).
- [27] S.-J. Huang and Z.-H. Zhou, "Multi-label learning by exploiting label correlations locally," in *Proc. 26th AAAI Conf. Artif. Intell.*, 2012, pp. 949–955.
- [28] Y. Gong, Q. Zhang, X. Han, and X. Huang, "Phrase-based hashtag recommendation for microblog posts," *Sci. China Inf. Sci.*, vol. 60, no. 1, 2017, Art. no. 12109.
- [29] M. Ghashami, D. J. Perry, and J. Phillips, "Streaming kernel principal component analysis," in *Proc. 19th Int. Conf. Artif. Intell. Statist.*, 2016, pp. 1365–1374.
- [30] X.-Z. Wu and Z.-H. Zhou, "A unified view of multi-label performance measures," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 3780–3788.
- [31] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *J. Mach. Learn. Res.*, vol. 9, pp. 1871–1874, 2008.
- [32] N. Boumal, B. Mishra, P.-A. Absil, and R. Sepulchre, "Manopt, a Matlab toolbox for optimization on manifolds," *J. Mach. Learn. Res.*, vol. 15, pp. 1455–1459, 2014.
- [33] M. Hollander, D. A. Wolfe, and E. Chicken, *Nonparametric Statistical Methods*. Wiley, 2013.
- [34] X. Mu, K. M. Ting, and Z.-H. Zhou, "Classification under streaming emerging new classes: A solution using completely random trees," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 8, pp. 1605–1618, 2017.
- [35] G. Tsoumakas and I. P. Vlahavas, "Random k-labelsets: An ensemble method for multilabel classification," in *Proc. 18th Eur. Conf. Mach. Learn.*, 2007, pp. 406–417.
- [36] Y. Zhu, K. M. Ting, and Z.-H. Zhou, "Multi-label learning with emerging new labels," in *Proc. IEEE 16th Int. Conf. Data Mining*, 2016, pp. 1371–1376.



Yue Zhu received the BSc degree in computer science and technology from Nanjing Normal University, Nanjing, China, in 2011. In the same year, he was admitted to further study at Nanjing University, Nanjing, China. Now, he is currently working toward the PhD degree and a member of the LAMDA Group. His research interests mainly include multi-instance learning, multi-label learning, multi-view learning, incremental learning, and novelty detection.



Kai Ming Ting received the PhD degree from the University of Sydney, Australia. After receiving the PhD degree, he worked with the University of Waikato, Deakin University, and Monash University. He joined Federation University, in 2014. He previously held visiting positions with Osaka University (Japan), Nanjing University (China), and the Chinese University of Hong Kong. His current research interests include mass estimation and mass-based approaches, ensemble approaches, and data stream data mining. He is an associate editor for the *Journal of Data Mining and Knowledge Discovery*. He co-chaired the Pacific-Asia Conference on Knowledge Discovery and Data Mining 2008. He has served as a member of program committees for a number of international conferences including ACM SIGKDD, IEEE ICDM, and ICML. His research projects are supported by grants from the Australian Research Council, US Air Force of Scientific Research (AFOSR/AOARD), and Australian Institute of Sport. Awards received include the Runner-up Best Paper Award in 2008 IEEE ICDM and the Best Paper Award in 2006 PAKDD.



Zhi-Hua Zhou (S'00-M'01-SM'06-F'13) received the BSc, MSc, and PhD degrees in computer science from Nanjing University, China, in 1996, 1998, and 2000, respectively, all with the highest honors. He joined the Department of Computer Science & Technology, Nanjing University, as an assistant professor, in 2001, and is currently a professor and standing deputy director of the National Key Laboratory for Novel Software Technology. He is also the founding director of the LAMDA Group. His research interests include mainly in artificial intelligence, machine learning, and data mining. He has authored the books *Ensemble Methods: Foundations and Algorithms* and *Machine Learning* (in Chinese), and published more than 150 papers in top-tier international journals or conference proceedings. He has received various awards/honors including the National Natural Science Award of China, the PAKDD Distinguished Contribution Award, the IEEE ICDM Outstanding Service Award, the Microsoft Professorship Award, etc. He also holds 22 patents. He is an executive editor-in-chief of the *Frontiers of Computer Science*, associate editor-in-chief of the *Science China Information Sciences*, action or associate editor of *Machine Learning*, the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, the *ACM Transactions on Knowledge Discovery from Data*, etc. He served as an associate editor-in-chief for the *Chinese Science Bulletin* (2008–2014), associate editor for the *IEEE Transactions on Knowledge and Data Engineering* (2008–2012), the *IEEE Transactions on Neural Networks and Learning Systems* (2014–2017), the *ACM Transactions on Intelligent Systems and Technology* (2009–2017), the *Neural Networks* (2014–2016), the *Knowledge and Information Systems* (2003–2008), etc. He founded ACML (Asian Conference on Machine Learning), served as advisory committee member for IJCAI (2015–2016), steering committee member for ICDM, PAKDD, and PRICAI, and chair of various conferences such as general co-chair of PAKDD 2014 and ICDM 2016, program co-chair of SDM 2013 and IJCAI 2015 Machine Learning Track, and area chair of NIPS, ICML, AAAI, IJCAI, KDD, etc. He is/was the chair of the IEEE CIS Data Mining Technical Committee (2015–2016), the chair of the CCF-AI(2012–), and the chair of the Machine Learning Technical Committee of CAAI (2006–2015). He is a foreign member of the Academy of Europe, and a Fellow of the ACM, AAAI, AAAS, IEEE, IAPR, IET/IEE, CCF, and CAAI.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.